PROYECTO FINAL: ENTORNOS DE DESARROLLO 🤗

Hecho por:

Sergio García-Morales Coca

Alejandro Guillén Plazuelo

Iván Labrador Eugenio.

Tutor:

Andrés Chillón Sesma

10/05/2026

Índice:

1. Resumen del proyecto (Página 3)
2. Introducción (Página 4)
3. Objetivos (Página 5)
4. Alcance del proyecto
    a. Requisitos funcionales y no funcionales (Pág. 6)
    b. Prototipo/Mockup (Pág. 7)
    c. Diagrama de Casos de Uso (Pág. 8)
    d. Diagramas de Clases de Análisis (Pág. 9-10)
    e. Diagrama de Entidad-Relación (Pág. 10)
    f. Diagrama de Clases de Diseño (Pág. 11-12)
    g. Fase de Testing (Pág. 12)
        i. Caja blanca
        ii. Caja negra
        iii. JUnit
5. Conclusiones (Pág. 13)
6. Bibliografía (Pág. 14)

1. Resúmen del proyecto

En este documento se va a pormenorizar en todos los pasos necesarios que hemos tenido que realizar para la elaboración de este proyecto, desde la idea fundamental hasta la creación de todos los diagramas empleados para la facilitación de, en un futuro, dar vida a este proyecto y poder crear un programa funcional de lo que, en principio, solo era una idea.

Este proyecto es la puesta en práctica de todo lo que hemos aprendido a lo largo de este curso escolar en la asignatura Entornos de Desarrollo y, por lo tanto, reflejará en detalle todo el conocimiento adquirido, no solo a nivel teórico, sino expresado de una forma muy cercana a la realidad del mundo laboral.

Tras el desarrollo de este proyecto, hemos llegado a la conclusión de la importancia de separar el análisis del diseño de una aplicación y, por lo tanto, de la gran ayuda que brinda el realizar unos diagramas detallados y centrados única y exclusivamente en su propósito, por ejemplo: la abstracción que ofrece un diagrama de casos de uso, permitiéndonos pensar en el diseño de código en una etapa más tardía del desarrollo; o la facilidad de organizar bases de datos utilizando el diagrama entidad-relación pertinente.

2. Introducción

¿Alguna vez has abierto la nevera y te has quedado mirando sin saber qué comer hasta que sonaba el pitido para cerrarla?

A menudo les pasa lo mismo a miles de jugadores; abren sus bibliotecas digitales y no saben a qué jugar, ya que se ven abrumados por cientos de empresas que buscan conseguir su atención.
Nuestra propuesta busca solucionar el problema de decidir. El usuario, al iniciar la aplicación, podrá ver un botón de jugar (junto a otros botones de configuración y accesibilidad). Este botón inicia una versión de prueba de un juego seleccionado aleatoriamente y permite al usuario jugarlo, valorarlo y dar feedback al desarrollador.

Mediante este sistema se forma una relación mutualista o simbiótica entre usuario y desarrollador: el usuario juega gratuitamente y descubre nuevos proyectos prometedores; mientras que el desarrollador se retroalimenta de las críticas y valoraciones de los jugadores sin pagar por beta testing.

Hemos decidido usar esta idea para el proyecto con dos objetivos en mente: el primero, solucionar el problema que se plantea en la pregunta del principio; el segundo, crear un entorno de descubrimiento y testeo de ideas para desarrolladores y jugadores que sea gratuito.

3. Objetivos

Nuestros principales objetivos con la realización de este proyecto se podrían desglosar en tres puntos principales:

1. Aprender. Conocíamos de antemano la densidad de este proyecto y decidimos aprovecharla para aumentar nuestros conocimientos, reforzando las flaquezas que teníamos.

2. Acercarnos a la vida laboral. Al iniciar este proyecto teníamos bastante claro que el desarrollo funcionaría de una forma similar a como ocurre en el mundo laboral.

3. Sacar todo nuestro potencial. Queríamos usar este proyecto para poner en práctica lo aprendido en el curso y demostrar nuestra disciplina.

4. Alcance del proyecto

a. Requisitos funcionales y no funcionales

========== REQUISITOS FUNCIONALES ==========

1. Permitir a los desarrolladores de los juegos subir una previsualización/demo a nuestra plataforma

2. Almacenar estadísticas útiles para desarrolladores. Ej: tiempo jugado, esquema de controles utilizado, etc.

3. Implementar elección aleatoria de los juegos subidos a la plataforma.

========== REQUISITOS NO FUNCIONALES ==========

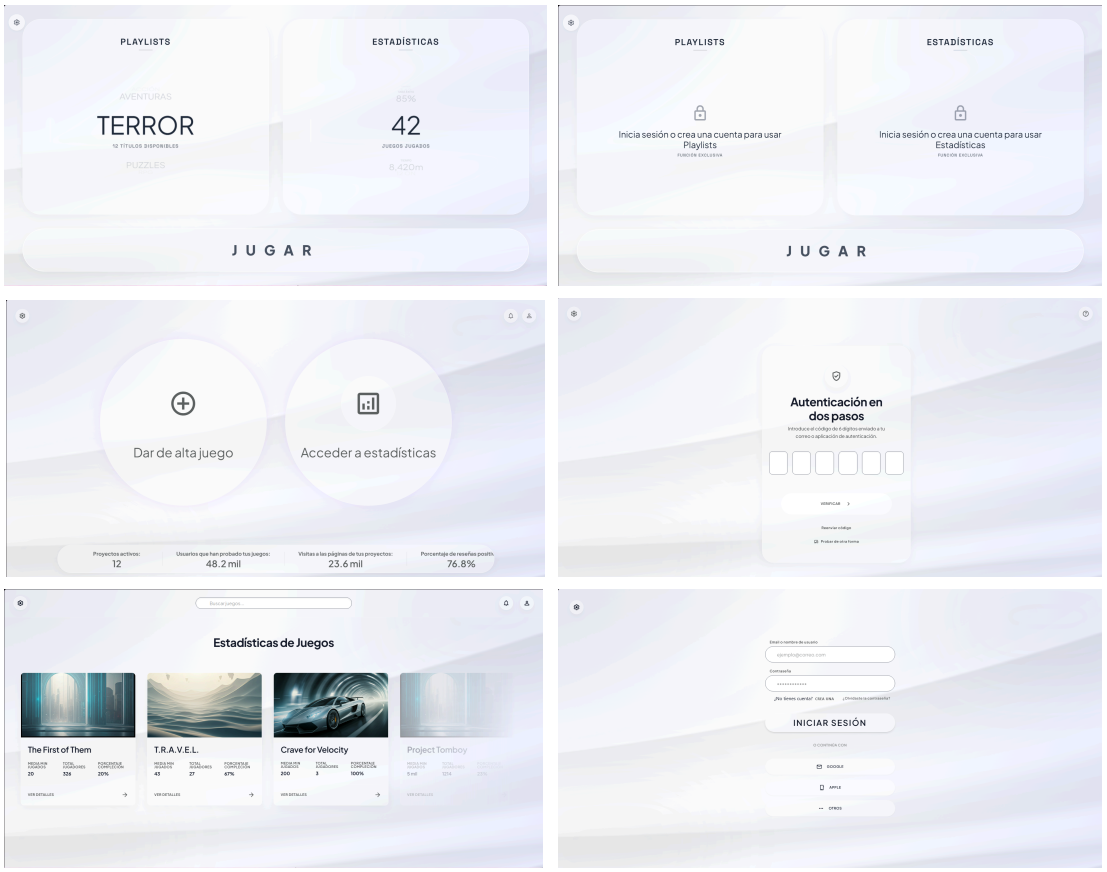
1. Diferentes interfaces para usuarios "normales" y usuarios "desarrollador".

2. Apartado de comentarios al terminar de probar cada juego, pudiendo dar feedback a los desarrolladores.

3. White/Blacklist de géneros de videojuegos para usuarios.

El primer paso para poder desarrollar una idea y empezar a darle forma es recoger los requisitos, tanto funcionales como no funcionales. En el caso de nuestro proyecto no son muchos, pero son más que suficientes para comenzar a dar cuerpo al concepto inicial que tenemos en mente.

b. Prototipo / Mockup



Gracias al prototipo, conseguimos dar el primer paso, alejándonos de la abstracción y empezando a concretar las necesidades técnicas y de diseño que necesitaremos para seguir adelante con nuestro proyecto.

Hemos podido saber cómo realizar todos los diagramas siguientes gracias a tener un prototipo donde concretamos todo lo que creíamos necesario para el correcto funcionamiento de nuestro programa.

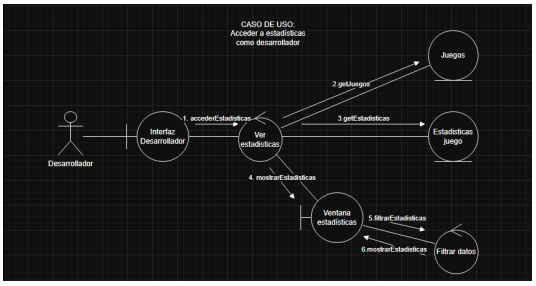
Hemos aprovechado la cantidad de tareas necesarias para el trabajo para experimentar con herramientas de IA especializadas en desarrollo de Front End. En nuestro caso hemos utilizado la IA generativa Stitch por medio de prompts.

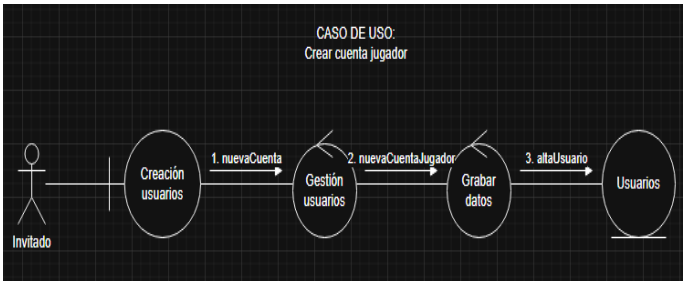
c. Diagrama de Casos de Uso

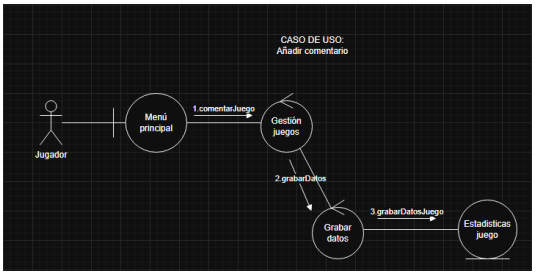
Si bien un diagrama de casos de uso sigue siendo una etapa muy abstracta del desarrollo, gracias al mismo somos capaces de empezar a separar posibles bloques de código que más adelante implementaremos.

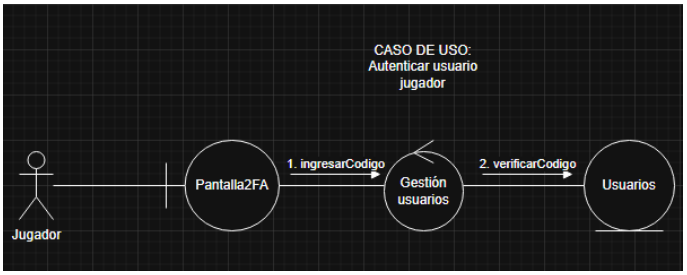
Al igual que con los prototipos, este diagrama nos permite la creación de siguientes diagramas y facilita, a rasgos generales, la organización para el desarrollo de la aplicación a nivel de código, por ejemplo: decidir qué miembro se hará cargo de implementar cada caso de uso en el momento de empezar a escribir código.

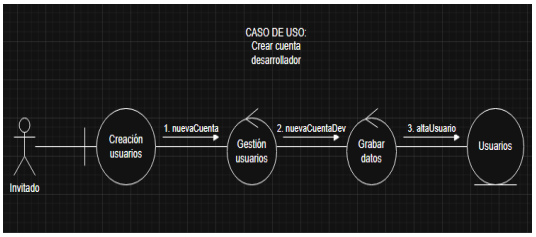
d. Diagrama de Clases de Análisis

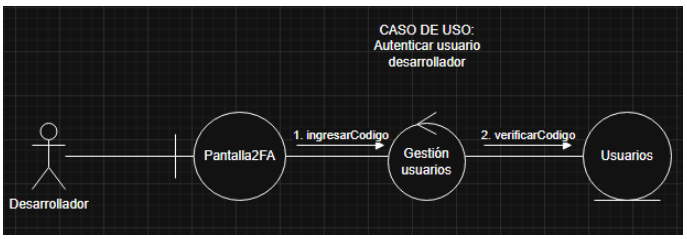


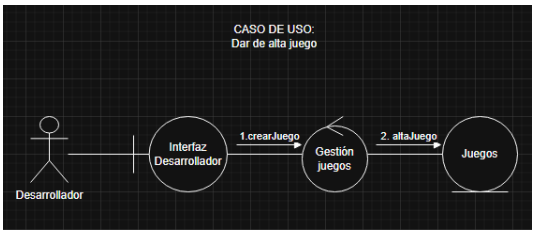


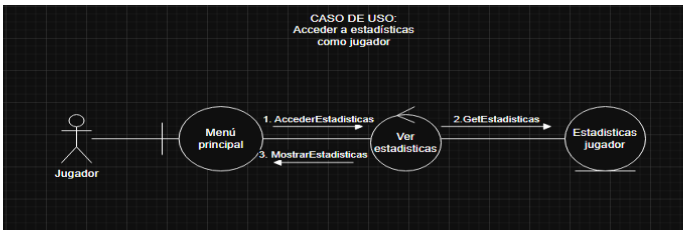


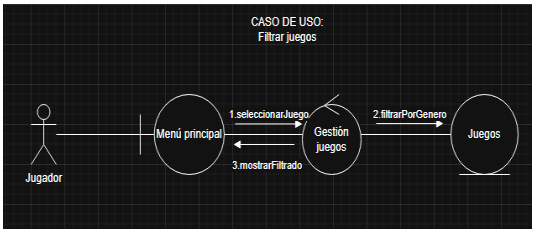


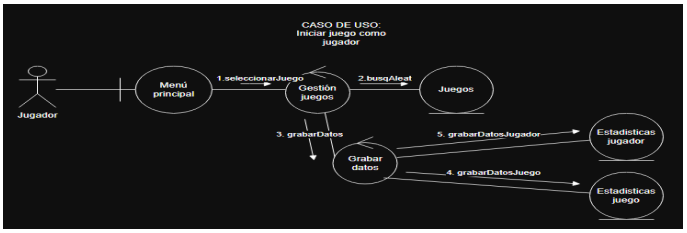


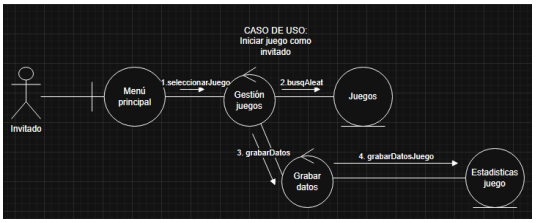


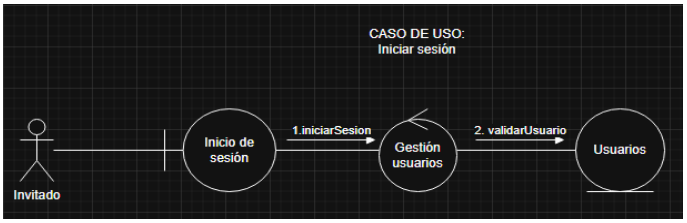


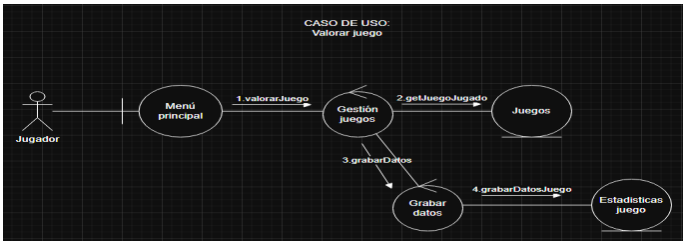






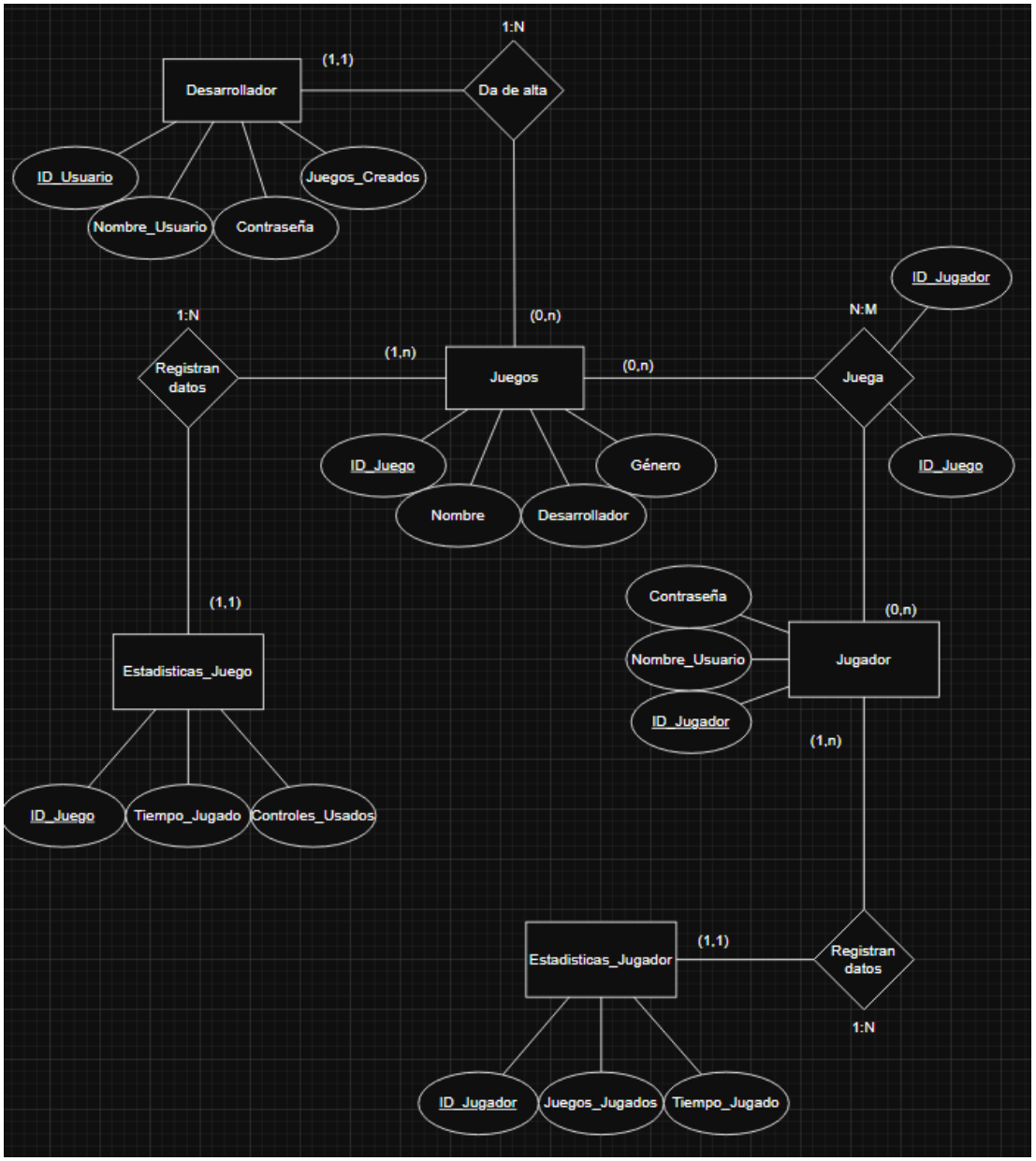






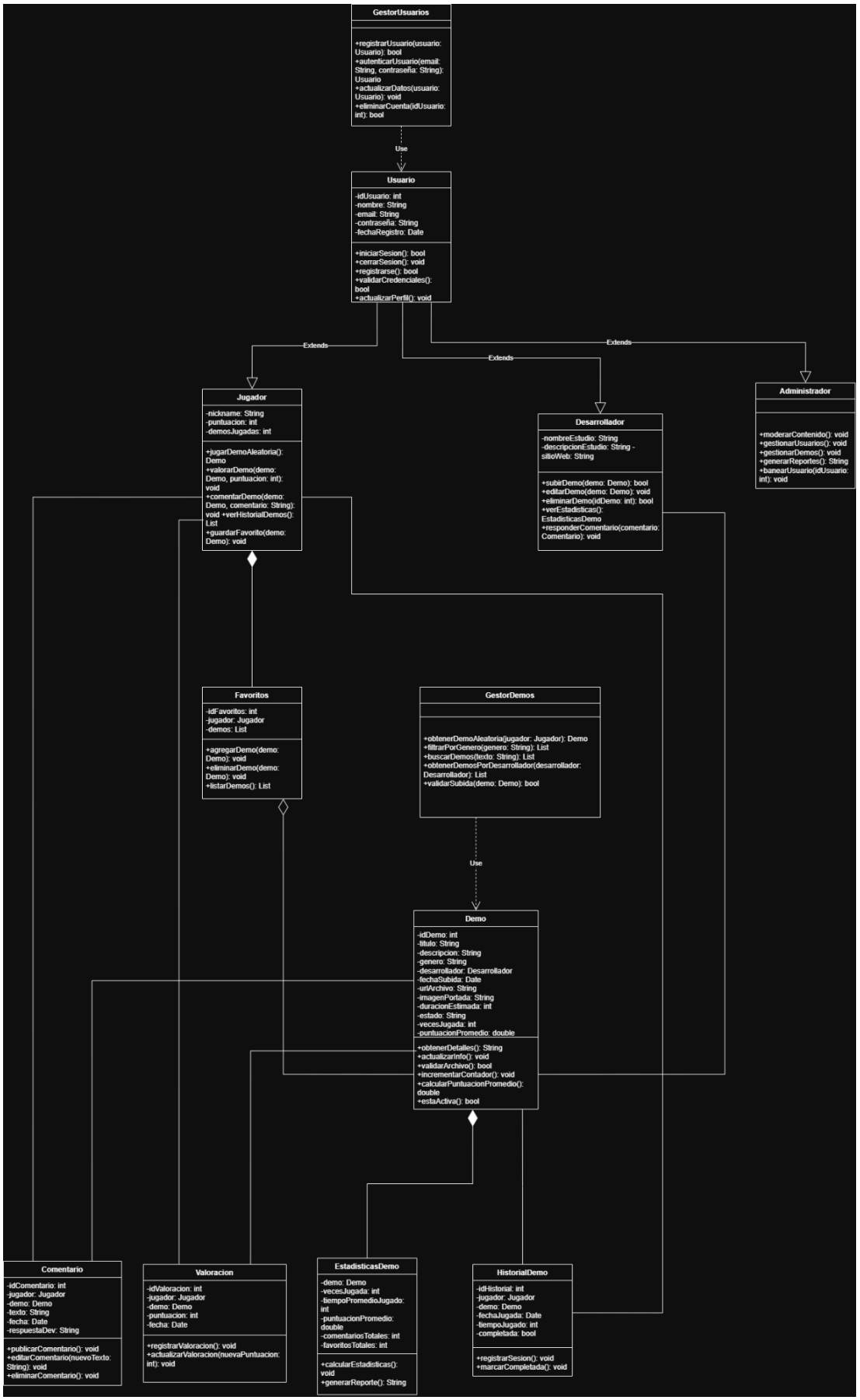
Nos encontramos ahora en una fase fundamental del análisis necesario para el desarrollo del proyecto. Los diagramas de clases de análisis nos brindan la posibilidad de pormenorizar en cada caso de uso. En ellos, detallamos el funcionamiento esperado de cada caso de uso y, aun siguiendo en un nivel abstracto, empezamos a ver partes tangibles del programa, como pueden ser interfaces o bases de datos.

e. Diagrama Entidad-Relación



Al utilizar bases de datos, es necesaria la elaboración de un diagrama entidad-relación para poder empezar a concretar qué datos guardaremos. Si bien no refleja la extensión real de nuestra base de datos, nos ofrece una visión inicial, la cual puede ser ampliada más adelante.

f. Diagrama de Clases de Diseño



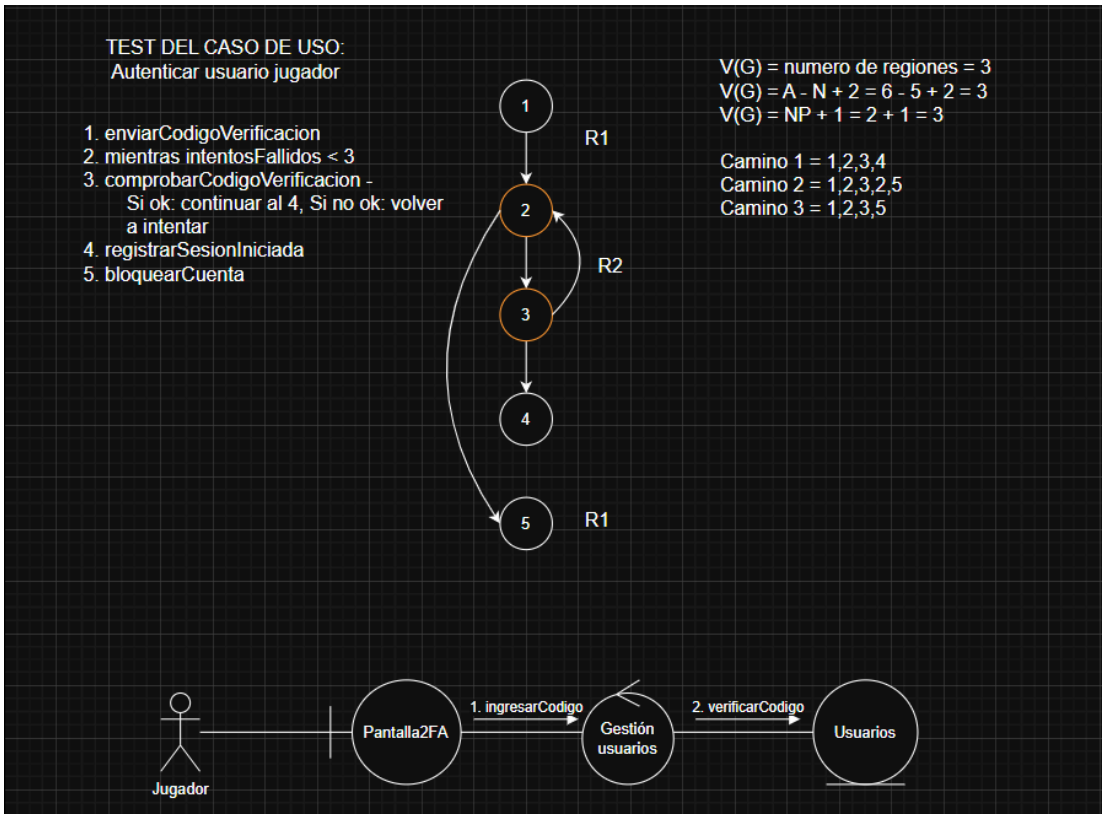
El diagrama de clases de diseño es el último paso antes de empezar a implementar código. Echando un simple vistazo ya empezamos a vislumbrar la lógica de programación necesaria.

La creación de este diagrama ha sido posible gracias al trabajo volcado en los diagramas previos, demostrando la importancia de los mismos y la facilidad que otorga realizar un buen análisis en las etapas tempranas del desarrollo.

g. Fase de Testing

Un proceso muy importante una vez empezamos a diseñar el código es diseñar, al mismo tiempo, unos tests robustos que no solo nos permitan probar lo ya diseñado, sino también pulir ideas para ampliar el diseño.

i. Caja blanca



Conociendo la estructura interna, el código fuente y la lógica de nuestra aplicación, el paso más lógico es hacer un grafo de flujos para cada caso de uso. Gracias a él podemos verificar el correcto funcionamiento técnico del código.

ii. Caja negra

Condición de entrada	Clases válidas	Clases no válidas
Código de verificación introducido por el usuario	Igual al código de verificación generado && longitud = 6 (4)	!= A código generado (3) longitud !=6 (3) No son números (3)
Intentos de verificación	intentos < 3 (2)	intentos > 3 (5)

Otros tipos de tests necesarios son los de caja negra. Con ellos podemos comprobar la funcionalidad externa de nuestro programa. Con este tipo de test no se pretende evaluar la estructura del código, sino las entradas y las salidas. Se centran en simular el comportamiento de un usuario.

iii. JUnit

```java
package junitproyectoed;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class UsuarioTest {

    @Test
    public void testValidarCredencialesCorrectas() {
        String codigoGenerado = "676942";
        String codigoUsuario = "676942";
        Usuario instance = new Usuario();

        boolean expResult = true;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }

    @Test
    public void testValidarCredencialesIncorrectas() {
        String codigoGenerado = "676942";
        String codigoUsuario = "499371";
        Usuario instance = new Usuario();

        boolean expResult = false;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }

    @Test
    public void testValidarCredencialesVacias() {
        String codigoGenerado = "";
        String codigoUsuario = "";
        Usuario instance = new Usuario();

        boolean expResult = false;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }

    @Test
    public void testValidarCredencialesNull() {
        String codigoGenerado = "";
        String codigoUsuario = "";
        Usuario instance = new Usuario();

        boolean expResult = false;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }

    @Test
    public void testValidarCredencialesLongitudIncorrectaMenor() {
        String codigoGenerado = "1829";
        String codigoUsuario = "123654";
        Usuario instance = new Usuario();

        boolean expResult = false;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }
    @Test
    public void testValidarCredencialesLongitudIncorrectaMayor() {
        String codigoGenerado = "182911";
        String codigoUsuario = "123654789";
        Usuario instance = new Usuario();

        boolean expResult = false;
        boolean result = instance.validarCredenciales(codigoGenerado, codigoUsuario);

        assertEquals(expResult, result);
    }

}
```

Activar Windows
Ve a Configuración para activar Windows.

Por último, pero no menos importante, necesitamos comprobar el correcto funcionamiento de los métodos implementados. Utilizando JUnit, hemos creado varios tests para el método credenciales(), lo cual nos permite no solo comprobar el correcto funcionamiento, sino también hacer predecibles los errores. La creación de este tipo de test es fundamental para el desarrollo dirigido por pruebas (test driven development).

5. Conclusiones

Tras concluir el proyecto, hemos reflexionado sobre qué nos ha aportado este. En primer lugar, hemos puesto en práctica todo lo aprendido durante el curso y hemos podido experimentar de primera mano algunos de los problemas lógicos y de diseño que a menudo se viven en proyectos profesionales. También hemos logrado comprender mejor los distintos diagramas de los que nos hemos servido para realizar la estructura lógica y conceptual de la aplicación.

Otro de los resultados que hemos tenido, a lo largo de todo el proyecto, ha sido darnos cuenta de la complejidad real que tienen las aplicaciones profesionales. Algo que pensábamos que tenía menos importancia en el desarrollo es el análisis. Gracias al proyecto nos hemos dado cuenta de que, en el mundo laboral, un proceso de desarrollo es algo más metódico y profundo que la mera escritura de código.

Para finalizar, el tercer objetivo cumplido ha sido el volcarnos por completo en este proyecto. Desde el primer día hemos dedicado un gran esfuerzo para no solo terminar el proyecto, también para refinarlo y hacer un trabajo del cual podamos estar orgullosos.

Si hay una cosa que hemos sacado en claro realizando este proyecto, es que somos más que capaces de entender no solo el diseño del código, el análisis que hay detrás también.

6. Bibliografía

Durante el desarrollo de este proyecto, nos hemos servido de los siguientes recursos:

1. Documentación subida por el profesor al aula virtual.

2. Herramienta de Inteligencia Artificial para creación de interfaces stitch.withgoogle.com. Usada para la creación de interfaces en el apartado 4.b. Prototipo/Mockup.

   Algunos ejemplos de prompts utilizados:

   "Queremos diseñar una aplicación que, al pulsar el botón jugar, te cargue un juego aleatorio de una base de datos. La interfaz es la siguiente:
   En la parte superior puedes encontrar los últimos juegos que has probado (cuando los juegas los "desbloqueas"). En las playlists un scrollable tipo ruleta en la cual te permita elegir distintos géneros de videojuego.
   A la derecha encuentras el mismo tipo de scrollable, pero esta vez te muestra las estadísticas de tus partidas.
   Las stats del jugador están en burbujas. Haz que se muestren como las playlists, con el título arriba y la correspondiente estadística debajo en pequeñito.
   Los juegos jugados tienen que ir encima de los minutos jugados y tiene que ser un grid con las carátulas de los videojuegos. "

   "Now we're going to create more screens for different options and users.
   A guest / unlogged user. Use the same template but replace the playlists and stats with a message that says "log in or create an account to use playlists / log in or create an account to see your stats". The rest keep it the same.
   A developer. Delete the "JUGAR" button and resize the playlists and estadìsticas areas. Now, these two are going to be round big buttons. The right one will display a title saying "Acceder a estadísticas"; the left one will say "dar de alta juego". Keep everything else the same."

   Para ver el resto de prompts usados:
   https://github.com/ProyectoED/AplicacionTestingVideojuegos/blob/main/Maquetas/Prompts%20usados%20Stitch.txt